



智能计算系统学习日志

从神经网络到全栈系统

作者: ZHY

时间: July 16, 2026

版本: 0.1



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

目录

第 1 章 智能计算系统认知地图	1
1.1 人工智能及其计算系统发展脉络	1
1.2 智能计算系统框架	1
第 2 章 神经网络基础	2
2.1 机器学习经典框架	2
2.2 线性回归模型	3
2.2.1 Representation	3
2.2.2 Evaluation	3
2.2.3 Optimization	3
2.3 神经网络	4
2.3.1 Representation	4
2.3.2 Evaluation	5
2.3.3 交叉验证	5
2.3.4 Optimization	5
第 3 章 深度学习	7
3.1 计算机视觉发展长廊	7
3.1.1 卷积神经网络的结构	7
3.1.1.1 卷积层	7
3.1.1.2 池化层	9
3.1.1.3 全连接层与 Softmax 输出层	9
3.1.2 卷积神经网络的优化	10
3.1.2.1 CNN 网络的前向传播	11
3.1.2.2 CNN 网络的反向传播	11
3.1.2.3 优化技巧	13
3.1.3 应用于图像分类的卷积神经网络	16
3.1.3.1 AlexNet ——深度学习的破冰者	16
3.1.3.2 VGG ——深度与简洁的极致统一	17
3.1.3.3 Inception-v1 (GoogLeNet) ——多尺度卷积的并行探索	17
3.1.3.4 BN-Inception ——批量归一化带来的训练革命	18
3.1.3.5 Inception-v3	19
3.1.3.6 ResNet	19

第 1 章 智能计算系统认知地图

内容提要

❑ 智能计算系统发展史

❑ 智能计算系统全流程视图

本课程本质上探讨一个问题：如何从无到有实现人工智能。

该问题源于一个关键预设：人工智能的实现是一个系统工程，今天大家却大部分仅关注算法问题。

- 新的时代对人工智能的需求勃发，新的系统被称为智能计算系统。
- 智能的核心是算法，算法运行在具体的机器上，机器的运行代价不容忽视。
- 想要在现实世界中设计优化并运用人工智能算法，必须了解整个智能计算系统。

1.1 人工智能及其计算系统发展脉络

纵观计算机发展百年史，每一代人工智能浪潮，总是经历着先是核心认知思路的改变，再到具体算法不断演进，配套的计算系统开始设计重组，最终人们的生活形成大的变化。

总结如下。

表 1.1: 三代智能计算系统对比

特征维度	第一代 (1980s)	第二代 (2010s)	第三代
核心认知	符号主义	连接主义	二者融合
方法论	形式逻辑、专家系统	深度学习、反向传播	探索中
计算载体	Prolog 机、LISP 机等专用计算机	GPU、AI 专用芯片 (TPU/NPU)	探索中
核心瓶颈	逻辑规则难以穷尽	算力与能耗瓶颈	探索中

1.2 智能计算系统框架

从系统层面观察一个大语言模型，其简要架构如下图所示。本课程核心目标就是打通智能系统全流程。

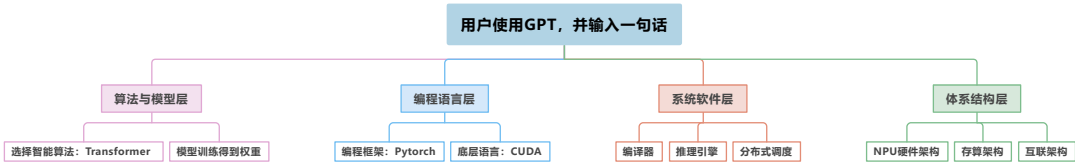


图 1.1: NPU 推理全栈流程图

第2章 神经网络基础

内容提要

- ❑ 线性回归模型
- ❑ 正则化思路

- ❑ 交叉验证方法
- ❑ 神经网络基础

本章内容看上去杂多，实则只是讨论了两个子问题。

- 以线性回归为例，讨论了机器学习的基本流程。
- 以神经网络为例，讨论了深度学习的基本流程。

二者同属一派，关系如下图所示。

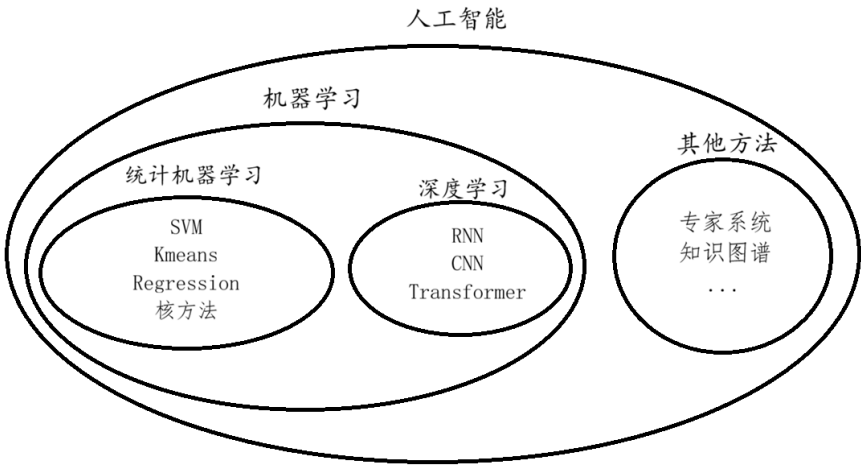


图 2.1: 机器学习、深度学习与人工智能的关系

2.1 机器学习经典框架

所有的机器学习算法本质上都是在某一输入空间和输出空间中，寻找一个函数模型，其可以最好地描述输入数据和输出数据的关系。因此所有的机器学习算法必然由三部分组成。

- **Representation:** 选择一个函数模型，作为输入和输出的映射关系。
- **Evaluation:** 选择一个评价函数，作为衡量模型好坏的标准。
- **Optimization:** 选择一个优化算法，作为寻找最优模型的手段。

上述分析思路我们称之为 **R-E-O** 框架。接下来，我们将以此为线索讨论两个经典模型。

2.2 线性回归模型

2.2.1 Representation

命题 2.1 (线性回归模型的输入输出空间)

线性回归模型在下述空间中被定义。

输入: $\mathbf{x} = [1, x_1, \dots, x_m]^T \in \mathbb{R}^{m+1}$

输出: $y \in \mathbb{R}$

数据集: $\mathcal{S} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$

核心假设:

- 输入 $\mathbf{x}$ 是随机变量 $\mathcal{X}$ 的采样结果
- 输出 y 是随机变量 $\mathcal{Y}$ 的采样结果
- $(\mathbf{x}, y)$ 是联合分布 $\mathcal{D}$ 的采样结果
- 数据集 $\mathcal{S}$ 是联合分布 $\mathcal{D}^n$ 的独立同分布采样结果

定义 2.1 (线性回归模型)

线性回归模型在函数空间 $\mathcal{F}$ 中试图寻找一个线性映射 f_w

$$\mathcal{F} = \{f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} \mid \mathbf{w} = [w_0, w_1, \dots, w_m]^T \in \mathbb{R}^{m+1}\} \quad (2.1)$$

对于给定的 $\mathbf{w}$, 有 $f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + \dots + w_m x_m$ 。

注 上述论述是线性回归模型的形式化定义。一言以蔽之, 线性回归模型假设 $\mathbf{x}$ 和 y 之间的关系是线性的, 并且通过参数 $\mathbf{w}$ 来描述这种线性关系。

2.2.2 Evaluation

在线性回归中, 评估函数衡量模型预测值 $f_{\mathbf{w}}(\mathbf{x})$ 与真实值 y 之间的差距。最常用的选择是均方误差 (Mean Squared Error, MSE)。

定义 2.2 (线性回归的损失函数)

给定数据集 $\mathcal{S} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, 线性回归模型的损失函数定义为:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \left(f_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 \quad (2.2)$$

等价地, 可以写成矩阵形式:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \quad (2.3)$$

其中 $\mathbf{X} \in \mathbb{R}^{n \times (m+1)}$ 为设计矩阵, $\mathbf{y} = [y^{(1)}, \dots, y^{(n)}]^T$ 。

注 MSE 作为评估函数的合理性源于其数学性质: 在误差服从均值为零的高斯分布假设下, 最小化 MSE 等价于极大似然估计。这一性质为线性回归提供了概率论层面的解释。

2.2.3 Optimization

线性回归的优化目标是 minimize 损失函数 $\mathcal{L}(\mathbf{w})$, 即求解:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \quad (2.4)$$

由于 $\mathcal{L}(\mathbf{w})$ 是凸函数, 存在闭式解 (解析解) 和数值解两种求解路径。

命题 2.2 (线性回归的闭式解)

令 $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = 0$ ，可得：

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (2.5)$$

该解被称为普通最小二乘（**Ordinary Least Squares, OLS**）估计量。



注 闭式解的求解需要 $\mathbf{X}^T \mathbf{X}$ 可逆。当特征数 m 很大 ($m > n$) 或存在多重共线性时，矩阵不可逆，此时必须引入正则化项。

在实际应用中，由于矩阵求逆运算复杂度很高，通常采用**梯度下降（Gradient Descent）**进行迭代求解。

命题 2.3 (线性回归: 梯度下降法)

从几何直观上理解，梯度是函数上升最快的方向，因此沿着梯度的反方向更新参数 $\mathbf{w}$ ，可以逐步逼近最优解：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \quad (2.6)$$

其中 η 为学习率，梯度计算公式为：

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y}) \quad (2.7)$$



注 从 R-E-O 框架审视线性回归：线性函数族完成了表示，MSE 完成了评估，而解析解或梯度下降完成了优化。三者合在一起，构成了一个有完整数学闭合形式的机器学习算法。

2.3 神经网络

如果说线性回归是最简单的机器学习模型，那么神经网络则代表了另一条截然不同的路线：通过组合基本非线性单元来逼近任意复杂函数。

2.3.1 Representation

定义 2.3 (人工神经元)

一个神经元是一个计算单元，接收输入向量 $\mathbf{x} \in \mathbb{R}^d$ ，输出标量值：

$$a = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (2.8)$$

其中 $\mathbf{w} \in \mathbb{R}^d$ 为权重向量， $b \in \mathbb{R}$ 为偏置， $\sigma(\cdot)$ 为激活函数（如 ReLU、Sigmoid、Tanh）。

**定义 2.4 (神经网络的前向传播)**

一个 L 层前馈神经网络通过逐层复合的方式定义函数：

$$f(\mathbf{x}) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}) \quad (2.9)$$

其中每一层 f_ℓ 是一个仿射变换 ($\mathbf{z}_\ell = \mathbf{W}_\ell \mathbf{a}_{\ell-1} + \mathbf{b}_\ell$) 后接激活函数 σ_ℓ 。

记第 ℓ 层的神经元数为 d_ℓ ，输入层 $d_0 = m$ ，输出层 $d_L = k$ 。则第 ℓ 层的参数矩阵为：

$$\mathbf{W}_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \quad \mathbf{b}_\ell \in \mathbb{R}^{d_\ell} \quad (2.10)$$

整个网络的参数集合为 $\theta = \{\mathbf{W}_\ell, \mathbf{b}_\ell\}_{\ell=1}^L$ 。



定理 2.1 (通用近似定理, Universal Approximation Theorem)

设 $\sigma(\cdot)$ 为一非常数、有界、单调递增的连续激活函数（如 Sigmoid 或 Tanh）。令 I_m 表示 m 维单位超立方体 $[0, 1]^m$, $C(I_m)$ 表示定义在 I_m 上的所有连续函数构成的空间。则对于任意目标函数 $f \in C(I_m)$ 和任意精度 $\varepsilon > 0$, 存在整数 N 、实数常数 $v_i, b_i \in \mathbb{R}$ 以及实向量 $\mathbf{w}_i \in \mathbb{R}^m$ ($i = 1, \dots, N$), 使得如下形式的单隐层前馈神经网络:

$$F(\mathbf{x}) = \sum_{i=1}^N v_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i) \quad (2.11)$$

可以作为 f 的近似函数, 满足:

$$|F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon, \quad \forall \mathbf{x} \in I_m \quad (2.12)$$



注 该定理表明, **宽度** (即单层神经元数) 足够大的前馈神经网络具有万能逼近能力。它说明神经网络的表示能力在理论上是完备的——任何连续函数都可以用单隐层网络以任意精度逼近。但该定理具有以下重要局限:

1. **未保证学习效率**: 未说明达到给定精度所需的最小神经元数 N 。
2. **未保证可学习性**: 不保证梯度下降等优化算法能找到它。
3. **未保证泛化性**: 需要额外的正则化与验证。

表示、评估和优化三个环节各自的理论边界, 决定了实际系统的性能上限。即使表示能力极强也无济于事。

2.3.2 Evaluation

神经网络的评估函数取决于任务类型, 通常采用极大似然估计框架进行统一理解。

定义 2.5 (神经网络中的损失函数)

根据任务类型, 常见的损失函数有:

- 回归任务: 均方误差 (MSE), $\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \|f(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)}\|_2^2$
- 二分类任务: 二元交叉熵 (Binary Cross-Entropy), $\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$
- 多分类任务: 交叉熵 (Cross-Entropy), $\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k y_j^{(i)} \log \hat{y}_j^{(i)}$



2.3.3 交叉验证

2.3.4 Optimization

不同于线性回归的凸优化问题, 神经网络的损失函数是高度非凸的, 不存在闭式解。优化依赖于**反向传播 (Backpropagation)**和梯度下降及其变体。

定义 2.6 (反向传播算法)

对于第 ℓ 层的参数, 梯度通过链式法则从输出层向输入层逐层传递:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell} = \boldsymbol{\delta}_\ell \cdot \mathbf{a}_{\ell-1}^T \quad (2.13)$$

其中 $\boldsymbol{\delta}_\ell = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_\ell}$ 为第 ℓ 层的误差项, $\mathbf{a}_{\ell-1}$ 为前一层激活值。

**定义 2.7 (神经网络的参数更新)**

采用随机梯度下降 (SGD) 或其自适应变体进行参数更新:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \quad (2.14)$$

其中 $\mathcal{B}$ 为 mini-batch, $B = |\mathcal{B}|$ 为批大小。

实际训练中常用更先进的优化器 (如 Adam、AdamW), 其核心是自适应调整每个参数的学习率。



命题 2.4 (神经网络优化的核心挑战)

神经网络的优化面临以下关键挑战：

- 非凸性：损失函数存在大量局部极小值和鞍点，梯度下降无法保证收敛到全局最优。
- 梯度消失/爆炸：在深层网络中，梯度逐层相乘可能导致数值不稳定，梯度消失使浅层无法更新，梯度爆炸导致训练发散。
- 过拟合：由于参数量巨大（大模型参数量可达千亿级别），神经网络极易过拟合，必须依赖正则化技术：

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \Omega(\theta) \quad (2.15)$$

其中 $\Omega(\theta)$ 为参数范数惩罚（L1/L2），Dropout、早停等同样为有效手段。



注 从 R-E-O 框架审视神经网络：网络的深度和宽度决定了 Representation，损失函数与评价指标共同构成 Evaluation，而反向传播 + 梯度下降则构成了 Optimization。

命题 2.5 (R-E-O 框架的统一视角)

线性回归和神经网络虽然是复杂度迥异的两个模型，但在 R-E-O 框架下呈现出统一的结构：

	Representation	Evaluation	Optimization
线性回归	线性函数 $\mathcal{F} = \{\mathbf{w}^T \mathbf{x}\}$	MSE	解析解 / 梯度下降
神经网络	非线性复合函数	交叉熵 / MSE	反向传播 + SGD/Adam

机器学习算法的演变本质上是这三部分各自独立演进与协同创新的历史：表示的复杂化（从线性到非线性）、评估任务的多元化（从回归到生成）、优化技术的精进。理解了三者之间的相互作用，就理解了机器学习的核心方法论——而这也正是《智能计算系统》课程的理论起点。



第3章 深度学习

内容提要

□ 用于图像处理的深度学习网络

□ 用于文本和语音处理的深度学习网络

课程的这一章节，实际上进入了漫长的深度学习发展史，从计算机视觉，自然语言处理（大语言模型）两条线索出发，梳理了一个学科发展长廊中的璀璨明珠。

3.1 计算机视觉发展长廊

计算机视觉的发展必须要从 CNN 卷积神经网络说起。

3.1.1 卷积神经网络的结构

卷积神经网络（Convolutional Neural Network, CNN）是计算机视觉任务的基石。其核心设计思想源于生物视觉系统的两个重要特性：**局部感受野**（Local Receptive Field）和**权重共享**（Weight Sharing）。与全连接网络（Fully Connected Network, FCN）相比，CNN 通过显式地编码图像的空间结构先验，大幅减少了参数数量，从而使得深层网络的训练成为可能。

3.1.1.1 卷积层

定义 3.1 (卷积神经网络的基本组成)

一个典型的卷积神经网络由若干基本功能层堆叠而成，按信息流动顺序可划分为以下四类核心组件：

- **卷积层（Convolutional Layer）**：通过滑动卷积核提取输入数据的局部空间特征，是 CNN 中参数共享与稀疏连接的实现载体。
- **池化层（Pooling Layer）**：对特征图进行下采样，起到降维、增大感受野和引入平移不变性的作用。
- **全连接层（Fully Connected Layer）**：将提取到的高层语义特征展平并进行非线性组合，完成最终的决策或回归任务。
- **Softmax 层**：将全连接层的输出分数映射为概率分布，常作为分类任务的输出端。

命题 3.1 (卷积层的数学形式)

设输入特征图为 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ ，输出特征图为 $\mathbf{Y} \in \mathbb{R}^{H' \times W' \times C'}$ ，卷积核权重为 $\mathbf{W} \in \mathbb{R}^{k \times k \times C \times C'}$ ，偏置为 $\mathbf{b} \in \mathbb{R}^{C'}$ ，则卷积操作可形式化为：

$$\mathbf{Y}_{i,j,c'} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \sum_{c=1}^C \mathbf{X}_{i+u,j+v,c} \cdot \mathbf{W}_{u,v,c,c'} + \mathbf{b}_{c'} \quad (3.1)$$

其中 k 为卷积核尺寸， C 与 C' 分别为输入与输出通道数。

注[CNN 的设计哲学] 在未采用 CNN 之前，人们设想将输出的每一个信息与输入的每一个信息通过全连接层相连。但是参数爆炸，无法训练。而 CNN 的诞生则源于一下的简单直觉。

- 图像中表示同一特征的像素往往相互邻接。
- 图像中出现过的特征往往会在其他区域以相似的统计特性出现。

而卷积核的设计正是基于上述直觉：通过局部感受野捕捉邻接像素的相关性，通过参数共享捕捉全局特征的重复性。

命题 3.2 (卷积层的设计技巧)

卷积层的实际应用效果不仅取决于其数学定义，还高度依赖于三个核心超参数：边界扩充（Padding）、卷积步长（Stride）和空洞率（Dilation）。三者分别从“边缘处理”、“采样密度”和“视野跨度”三个维度调控特征图的输出特性。

名称	作用	具体做法
Padding	防止边缘信息丢失，控制输出尺寸	在输入特征图外侧填充零值或其他常数，再进行卷积
Stride	控制卷积核滑动的步长，实现下采样	每移动 s 个像素进行一次卷积运算
Dilation	在不增加参数量的前提下扩大感受野	在卷积核各元素间插入 0（空洞）

引入上述超参数后，卷积层输出尺寸的通用计算公式为：

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - [k + (k - 1)(d - 1)]}{\text{stride}} \right\rfloor + 1 \tag{3.2}$$

其中 k 为原始卷积核尺寸， d 为空洞率（ $d = 1$ 时退化为标准卷积，括号项还原为 k ）。

注[卷积神经网络的训练参数] **卷积层的可训练参数总结：** 对于一个卷积层，其可训练参数包括卷积核权重和偏置项，总量为：

$$\text{参数量} = (k^2 \times C_{\text{in}} + 1) \times C_{\text{out}} \tag{3.3}$$

其中 C_{in} 和 C_{out} 分别为输入和输出通道数。值得注意的是，该参数量与输入特征图的空间尺寸 $H_{\text{in}} \times W_{\text{in}}$ 完全无关——这正是参数共享机制带来的核心优势。由于偏置项 +1 相对于权重项 $k^2 \times C_{\text{in}}$ 通常可忽略，故常简化记为 $O(k^2 \cdot C_{\text{in}} \cdot C_{\text{out}})$ 。

命题 3.3 (深度可分离卷积)

深度可分离卷积（Depthwise Separable Convolution, DSC）将标准卷积分解为两个连续步骤，在保持相近表征能力的前提下显著降低参数量与计算量，是 MobileNet、Xception、EfficientNet 等轻量化网络的核心构建模块。

- **深度卷积（Depthwise Convolution）**——逐通道空间滤波。为每个输入通道独立分配一个 $k \times k$ 卷积核，通道间不发生混合，输出为 $\mathbf{Y}_{\text{dw}} \in \mathbb{R}^{H' \times W' \times C}$ 。参数量与计算量均为 $k^2 \times C \times H' \times W'$ 。
- **逐点卷积（Pointwise Convolution）**——跨通道特征组合。采用 1×1 卷积对深度卷积的输出进行通道维度的线性组合，生成 C_{out} 个输出通道。参数量与计算量均为 $C \times C_{\text{out}} \times H' \times W'$ 。

深度可分离卷积的总参数量与标准卷积之比为：

$$\frac{\text{Param}_{\text{DSC}}}{\text{Param}_{\text{std}}} = \frac{k^2 C + C C_{\text{out}}}{k^2 C C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2} \tag{3.4}$$

取 $k = 3$ 、 $C_{\text{out}} = 512$ 时，参数量约为标准卷积的 $1/512 + 1/9 \approx 11\%$ ，计算量缩减比例相同。

	标准卷积	深度可分离卷积	缩减比例
参数量	$k^2 C C_{\text{out}}$	$k^2 C + C C_{\text{out}}$	$\approx 1/k^2$ (C_{out} 较大时)
计算量	$k^2 H' W' C C_{\text{out}}$	$k^2 H' W' C + H' W' C C_{\text{out}}$	同上

注 深度可分离卷积的有效性依赖于“空间特征提取与通道特征组合可解耦”这一假设，该假设在自然图像中大致成立。当通道间存在高度复杂的非线性交互时（如注意力机制），标准卷积仍具优势。现代高效架构（如 MobileNetV2、EfficientNet）在此基础上引入倒置残差结构与线性瓶颈作为补充。

注[与 R-E-O 框架的关联] 深度可分离卷积是纯粹的表示（Representation）层面改进——改变了网络层的参数化结构，未改变评估准则或优化方法。表示的效率优化是模型走向实用化的关键驱动力。

3.1.1.2 池化层

池化层（Pooling Layer）是卷积神经网络中紧随卷积层之后的标准组件，其本质是一种固定的下采样算子——它不含可训练参数，仅通过确定的数学规则对特征图进行空间维度的压缩。池化层的设计动机源于三个层面的考量：降低计算复杂度、扩大感受野、以及引入对轻微形变的鲁棒性。

定义 3.2 (池化操作)

池化层将输入特征图 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ 划分为若干不重叠（或可重叠）的局部窗口 $\mathcal{N}_{i,j}$ ，对每个窗口内的激活值进行聚合统计，输出压缩后的特征图 $\mathbf{Y} \in \mathbb{R}^{H' \times W' \times C}$ 。设池化窗口尺寸为 $p \times p$ ，步长为 s ，则输出尺寸满足：

$$H' = \left\lfloor \frac{H-p}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W-p}{s} \right\rfloor + 1 \quad (3.5)$$

通道数 C 在池化过程中保持不变——池化独立作用于每个通道。

命题 3.4 (池化的主要类型)

按聚合函数的不同，池化层可分为两类经典变体：

名称	数学定义	特性	典型用途
最大池化	取局部窗口内的最大值作为输出	对显著特征敏感	主流 CNN 的默认选择
平均池化	取局部窗口内所有值的均值作为输出	保留背景信息	全局池化的基础

- 最大池化设计哲学：在一个局部区域内，最强烈的激活信号最能代表该区域的特征。可以抑制噪声。

注[池化层的核心作用] 池化层的贡献可归纳为以下三个层面：

1. **降维与计算量压缩**：池化操作使特征图的空间尺寸减半（典型配置 $p=2, s=2$ ）
2. **增大感受野**：后续卷积层的一个卷积核能覆盖输入图像上更大的空间范围，有利于高层语义特征的提取。
3. **平移不变性**：最大池化对局部区域内的微小位移具有不敏感性。若一个特征在窗口内任意位置出现，池化后都会被“浓缩”到同一个输出位置。

注[池化的“等价性”辨析] 需注意区分两个概念：

- **平移等变性 (Equivariance)**：输入平移 $\rightarrow$ 输出平移（卷积层具备）。特征的位置信息被保留，只是发生了相应的位移。
- **平移不变性 (Invariance)**：输入平移 $\rightarrow$ 输出不变（池化层具备）。特征的具体位置信息被丢弃，只保留“该特征是否出现”。

池化层的平移不变性，靠的不是单层窗口的大范围容忍，而是通过层层累积的“粗粒化”，将底层几像素的位移在顶层稀释为可忽略的误差。卷积提供的是“位置感知”，池化提供的是“位置鲁棒性”。两者互补，共同构成了 CNN 对空间结构的建模能力。

3.1.1.3 全连接层与 Softmax 输出层

在卷积层与池化层完成对输入图像的特征提取与空间压缩之后，网络需要将这些分布式的高层语义特征汇聚为最终的决策输出。这一职责由**全连接层**与**Softmax 输出层**共同承担——前者负责特征的全局组合与高阶抽象，后者负责将组合结果翻译为直观的概率分布。

定义 3.3 (全连接层的数学形式)

全连接层（Fully Connected Layer, FC）将前层输出的多维特征图展平（Flatten）为一维向量 $\mathbf{a} \in \mathbb{R}^D$ ，并通过仿射变换实现特征的非线性组合：

$$\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{D' \times D}, \quad \mathbf{b} \in \mathbb{R}^{D'} \quad (3.6)$$

其中 D 为输入特征维度， D' 为输出特征维度。全连接层的核心特性在于：输出层的每一个神经元都与输入层的所有神经元相连接，即“全连接”之名由此而来。

全连接层在 CNN 中扮演着“特征决策者”的角色——它将卷积层提取的局部特征（边缘、纹理、部件）进行全局性的加权组合，形成完整的语义判断。但是却由于参数量巨大，伴随着过拟合，参数爆炸等风险。因此，计算机视觉中另一种可能的处理方法是全局平均池化。

全局平均池化 = 对每个特征图取全局均值，把“空间位置”全部抹平，每个通道只留下一个数，以此替代全连接层的海量参数。

定义 3.4 (全局平均池化)

全局平均池化 (Global Average Pooling, GAP) 对特征图 $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ 的每个通道进行全局平均：

$$\mathbf{Y}_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W \mathbf{X}_{i,j,c}, \quad c = 1, \dots, C \tag{3.7}$$

输出为 $\mathbf{Y} \in \mathbb{R}^C$ 。GAP 将每个通道的空间信息压缩为单一标量，使得网络可接受任意尺寸的输入，且参数量降为零——其本质是“强制性的特征选择”：每个通道对应一种语义模式，其全局平均响应强度直接反映该模式在整张图像中的显著性。

注[全连接层 vs 全局平均池化]

对比维度	全连接层	全局平均池化
参数量	$D \times D'$ (数百万至数千万)	0
可解释性	权重难以可视化	每个通道对应一类语义响应
典型应用	AlexNet、VGG	ResNet、DenseNet、MobileNet

定义 3.5 (Softmax 层)

在分类任务中，全连接层（或 GAP）输出的得分向量 $\mathbf{z} \in \mathbb{R}^K$ （其中 K 为类别数）被称为 **Logits**，其取值可为任意实数，缺乏直观的概率解释。Softmax 层将 Logits 映射为概率单纯形上的分布：

$$\hat{y}_i = \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K \tag{3.8}$$

该映射具有三条优良性质：

1. 非负性： $\hat{y}_i > 0$ ，符合概率的公理化定义。
2. 归一性： $\sum_{i=1}^K \hat{y}_i = 1$ ，保证输出为一个合法的概率分布。
3. 保序性：Softmax 是单调递增函数，得分越高者概率越大，分类决策不受影响。

命题 3.5 (交叉熵损失)

Softmax 层通常与交叉熵损失 (Cross-Entropy Loss) 配合使用。设真实标签的独热编码为 $\mathbf{y} \in \{0, 1\}^K$ （其中仅正确类别处为 1，其余为 0），预测概率分布为 $\hat{\mathbf{y}}$ ，则交叉熵损失定义为：

$$\mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^K y_i \log \hat{y}_i = - \log \hat{y}_{\text{true}} \tag{3.9}$$

直观上：当模型对正确类别给出的概率越接近 1 时，损失趋近于 0；当概率趋近于 0 时，损失趋近于正无穷。交叉熵直接度量了两个概率分布之间的“距离”。

3.1.2 卷积神经网络的优化

神经网络的优化与线性回归等凸优化问题有着本质区别。深度网络的损失函数是高度非凸的，不存在闭式解析解，核心依赖于反向传播 (Backpropagation) 计算梯度，并结合梯度下降及其变体进行参数迭代更新。

命题 3.6 (优化的核心挑战)

深度神经网络的优化面临以下三重困境：

- 非凸性：损失函数存在大量局部极小值和鞍点。鞍点（Saddle Point）尤为棘手——其梯度为零，但既非极大也非极小，SGD 在此处会陷入停滞。高维空间中鞍点的数量远多于局部极小值，是深度网络优化的主要障碍。
- 梯度不稳定：在深层网络中，梯度逐层相乘（链式法则）可能导致数值不稳定。梯度消失（Gradient Vanishing）使浅层参数几乎不更新；梯度爆炸（Gradient Explosion）则导致参数更新发散。
- 过拟合风险：参数量远超样本量，网络极易记忆训练集噪声而非学习泛化规律。

3.1.2.1 CNN 网络的前向传播

前向传播是信息在 CNN 中从输入到输出的逐层流动过程。图像数据依次经过特征提取、特征整合与分类决策三个阶段，最终输出类别概率分布。

命题 3.7 (CNN 分类器的完整前向传播)

一个用于图像分类的 CNN，其前向传播路径可分解为三个阶段：

1. 特征提取阶段（卷积与池化）：输入图像 $\mathbf{X} \in \mathbb{R}^{H \times W \times 3}$ 依次经过 n 组“卷积 + 池化”模块，逐层降低空间分辨率、增加通道数，最终输出高层语义特征图：

$$\mathbf{X} \xrightarrow{\text{Conv}_1 \rightarrow \text{Pool}_1 \rightarrow \dots \rightarrow \text{Conv}_n \rightarrow \text{Pool}_n} \mathbf{A} \in \mathbb{R}^{h \times w \times C} \quad (3.10)$$

其中单个卷积层的前向计算为（第 ℓ 层输入为 $\mathbf{A}_{\ell-1}$ ）：

$$\mathbf{Z}_\ell = \mathbf{W}_\ell * \mathbf{A}_{\ell-1} + \mathbf{b}_\ell, \quad \mathbf{A}_\ell = f_\ell(\mathbf{Z}_\ell) \quad (3.11)$$

* 表示卷积运算， f_ℓ 为激活函数（通常为 ReLU）。

2. 特征整合阶段（全连接层）：将特征图 $\mathbf{A}$ 展平或全局平均池化为一维向量 $\mathbf{a} \in \mathbb{R}^D$ ，经全连接层映射为类别得分：

$$\mathbf{A} \xrightarrow{\text{Flatten / GAP}} \mathbf{a} \in \mathbb{R}^D \xrightarrow{\mathbf{W}_{fc}\mathbf{a} + \mathbf{b}_{fc}} \mathbf{z} \in \mathbb{R}^K \quad (3.12)$$

其中 K 为类别总数， $\mathbf{z}$ 为各类别的未归一化得分（Logits）。

3. 概率输出阶段（Softmax）：将得分向量 $\mathbf{z}$ 映射为概率单纯形上的分布：

$$\mathbf{z} \xrightarrow{\text{Softmax}} \hat{\mathbf{y}} \in \Delta^{K-1}, \quad \hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (3.13)$$

完整路径可概括为：

$$\mathbf{X} \xrightarrow{\text{卷积 + 池化} \times n} \mathbf{A} \xrightarrow{\text{展平/全局池化}} \mathbf{a} \xrightarrow{\text{全连接}} \mathbf{z} \xrightarrow{\text{Softmax}} \hat{\mathbf{y}} \quad (3.14)$$

训练阶段， $\hat{\mathbf{y}}$ 与真实标签 $\mathbf{y}$ （one-hot 编码）计算交叉熵损失：

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^K y_i \log \hat{y}_i \quad (3.15)$$

3.1.2.2 CNN 网络的反向传播

反向传播是链式法则在神经网络中的具体实现。它从损失函数出发，将误差信号逐层向输入端传递，计算损失对每一层参数的梯度，为参数更新提供依据。

命题 3.8 (CNN 反向传播：核心公式与操作)

符号约定： $\mathbf{A}_{\ell-1}$ ：输入 ($H_{\ell-1} \times W_{\ell-1} \times C_{\ell-1}$)， $\mathbf{W}_\ell$ ：卷积核 ($k \times k \times C_{\ell-1} \times C_\ell$)， $\mathbf{b}_\ell$ ：偏置 (C_ℓ)， $\mathbf{Z}_\ell$ ：卷积输出， $\delta_\ell = \partial \mathcal{L} / \partial \mathbf{Z}_\ell$ ：误差项。超参数：步长 s ，填充 p 。

3.1.2.2.1 输出层 (Softmax + 交叉熵)

$$\delta_{\text{out}} = \hat{\mathbf{y}} - \mathbf{y}$$

其中 $\hat{\mathbf{y}}$ 为预测概率向量, $\mathbf{y}$ 为真实标签的 one-hot 向量, δ_{out} 为输出层误差项。

3.1.2.2.2 全连接层

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{fc}}} = \mathbf{a} \cdot \delta^T, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_{\text{fc}}} = \delta, \quad \delta_{\text{prev}} = \mathbf{W}_{\text{fc}} \cdot \delta$$

其中 $\mathbf{a}$ 为全连接层输入向量, δ 为上游误差, $\mathbf{W}_{\text{fc}}$ 为权重矩阵, $\mathbf{b}_{\text{fc}}$ 为偏置。

3.1.2.2.3 池化层

- 最大池化: 误差只传回前向传播时窗口内最大值所在位置, 其余位置置零。
- 平均池化: 误差在窗口内均匀分配, 即 $\delta_{\text{prev}} = \frac{1}{p^2} \delta_{\text{pool}}$, 其中 p 为池化窗口尺寸。

3.1.2.2.4 卷积层 (核心)

3.1.2.2.4.1 偏置梯度

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_{\ell}} = \sum_{i,j} \delta_{\ell}[i, j, :]$$

即对误差张量的空间维度 (高度 i 和宽度 j) 求和。由于偏置梯度的个数等于卷积核的个数, 每一个卷积核作用在输入特征图上得到一个输出特征图, 因此每一个卷积核与一个误差矩阵对应, 对该误差矩阵求和即得到该卷积核的偏置梯度。

3.1.2.2.4.2 权重梯度 直观理解: 权重来源于卷积核。当卷积核在输入特征图上不断游走时: 每移动一次, 每一个权重会对应到一个输入特征图中的元素, 然后卷积, 形成一个输出元素。

于是权重梯度就等于 sum (该权重作用过的每一个输入元素 * 作用时形成的输出元素的梯度)

当考虑张量 (batch 和输入频道有多个) 时, 只要一一对应即可。

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\ell}[u, v, c, c']} = \sum_{i,j} \delta_{\ell}[i, j, c'] \cdot \mathbf{A}_{\ell-1}[is + u - p, js + v - p, c]$$

其中:

- u, v : 卷积核内的空间位置 ($0 \leq u, v < k$)
- c : 输入通道索引
- c' : 输出通道索引
- i, j : 输出特征图的空间位置
- $is + u - p, js + v - p$: 输入特征图上对应的位置

3.1.2.2.4.3 误差回传 直观理解: 一个元素通过多个卷积核进行卷积计算, 并形成多个输出特征图时: 它直接对应多个卷积核中的多个权重, 并通过计算, 得到多个输出特征图的对应元素。

一个输入元素对应多个权重, 每个权重唯一对应一个输出元素。所以将所有输出元素的梯度乘上对应权重并求和, 便得到输入元素的梯度。

$$\delta_{\ell-1}[x, y, c] = \sum_{c'} \sum_{\substack{i, j \\ (x, y) \in \text{window}(i, j)}} \delta_{\ell}[i, j, c'] \cdot \mathbf{W}_{\ell}[x - is + p, y - js + p, c, c']$$

其中：

- x, y : 输入特征图上的位置
- $\text{window}(i, j)$ 表示输出位置 (i, j) 的卷积窗口所覆盖的输入区域
- 条件 $(x, y) \in \text{window}(i, j)$ 等价于：

$$is - p \leq x < is - p + k, \quad js - p \leq y < js - p + k$$

- $x - is + p, y - js + p$: 输入位置 (x, y) 在卷积核中的相对位置

一句话：误差回传 = 把每个输出位置的误差”按权重”分配回所有看到它的输入位置。一个输入位置若被多个输出位置看到，则累加所有贡献。

3.1.2.2.5 激活函数（ReLU） 已在上一章介绍。等价地，逐元素写为：

$$\delta_{\ell}[i, j, c] = \begin{cases} \delta_{\ell}^{\text{upstream}}[i, j, c], & \mathbf{Z}_{\ell}[i, j, c] > 0 \\ 0, & \mathbf{Z}_{\ell}[i, j, c] \leq 0 \end{cases}$$

即只有前向传播时激活（正值）的位置，误差才能继续回传。

命题 3.9 (梯度消失与爆炸的根源)

由误差递推式可知，当网络加深时，梯度由一系列矩阵相乘构成：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\ell}} \propto \prod_{j=\ell}^{L-1} (\mathbf{W}_{j+1}^T \cdot \text{diag}(f'_j)) \quad (3.16)$$

若矩阵的谱范数 > 1 ，梯度指数级增长（爆炸）；若 < 1 ，梯度指数级衰减（消失）。

- **Sigmoid/Tanh**: 导数值域为 $(0, 1]$ ，连乘后极易趋近于 0，是梯度消失的主要诱因。
- **ReLU**: 正半轴导数为 1，负半轴为 0，在一定程度上缓解了梯度消失，但引入了“神经元死亡”问题（负半轴永不激活）。

在 CNN 中，若特征提取阶段的卷积层无法获得有效梯度，网络将退化为“仅全连接层可学、卷积层停滞”的状态——这正是 Xavier/Kaiming 初始化、Batch Normalization 等技巧试图解决的核心问题。

3.1.2.3 优化技巧

参数初始化

参数初始化是训练深度网络的第一步，也是最容易被忽视却至关重要的环节。初始化不当会导致梯度消失或爆炸，使网络从一开始就无法有效学习。

命题 3.10 (Xavier 初始化)

Xavier 初始化（Glorot 初始化）由 Glorot & Bengio (2010) 提出，其设计目标是使每一层的输入和输出方差保持一致，从而避免信号在前向传播中过度放大或衰减。其核心思想基于方差一致性原则：

- 适用激活函数：Sigmoid、Tanh 等关于原点对称、在零点附近近似线性的激活函数。
- 初始化方式：从均匀分布或正态分布中采样，满足：

$$\text{Uniform: } \mathbf{W} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right) \quad (3.17)$$

$$\text{Normal: } \mathbf{W} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \quad (3.18)$$

其中 n_{in} 和 n_{out} 分别为该层的输入和输出神经元数量。

Xavier 初始化的推导基于线性激活假设：若 $\text{Var}(x) = \text{Var}(y)$ ，则需要 $\text{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$ 。这一策略使得信号在前向传播和反向传播中均能保持稳定的方差尺度。

命题 3.11 (Kaiming 初始化)

Kaiming 初始化（何恺明初始化）由 He et al. (2015) 提出，专门针对 ReLU 激活函数及其变体设计。由于 ReLU 在负半轴输出为 0，会截断约一半的激活值，使得输出的方差约为输入方差的一半。为补偿这一衰减，Kaiming 初始化将方差放大一倍：

- 适用激活函数：ReLU 及其变体（Leaky ReLU、PReLU 等）。
- 初始化方式：

$$\mathbf{W} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \quad (3.19)$$

或从均匀分布采样：

$$\mathbf{W} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right) \quad (3.20)$$

初始化方法	适用激活函数	方差设置
Xavier	Sigmoid、Tanh	$\text{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$
Kaiming	ReLU 及其变体	$\text{Var}(W) = 2/n_{\text{in}}$

Kaiming 初始化是 ResNet 等极深网络能够成功训练的关键因素之一——它确保了在上百层的网络中，梯度仍能稳定传播。

注[初始化方法的选择原则] 实践中的通用准则：

- 使用 ReLU 激活函数 → 默认选择 Kaiming 初始化。
- 使用 Sigmoid/Tanh → 选择 Xavier 初始化。
- 现代框架（PyTorch、TensorFlow）的默认初始化通常是 Kaiming Uniform，因其对 ReLU 系网络最为友好。

优化器

参数初始化后，需通过优化器（Optimizer）迭代更新参数以降低损失。优化器的选择直接影响收敛速度与最终性能。以下按历史演进梳理各类优化算法。

命题 3.12 (批量梯度下降与随机梯度下降)

批量梯度下降（**Batch Gradient Descent**）每次更新使用全部训练数据计算梯度，梯度准确但计算开销巨大，且易陷入局部极小值。

随机梯度下降（**Stochastic Gradient Descent, SGD**）每次更新仅使用单个样本计算梯度，更新频繁且引入了随机性，有助于跳出局部极小。然而，其梯度估计存在较大方差，收敛路径震荡剧烈。

小批量随机梯度下降（**Mini-batch SGD**）是两者的折中，每次从训练集中随机抽取 B 个样本（通常 $B = 32, 64, 128, 256$ ）计算梯度：

$$\mathbf{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}_{t-1}) \quad (3.21)$$

参数更新为：

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \mathbf{g}_t \quad (3.22)$$

其中 η 为学习率。Mini-batch SGD 兼顾了计算的稳定性和效率，是深度学习的标准实践。

命题 3.13 (动量法 (Momentum))

SGD 在峡谷形曲面（一个方向陡峭、另一方向平缓）中易产生剧烈震荡。动量法引入“惯性”概念，将历史梯度的指数加权平均纳入更新：

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \mathbf{v}_t \quad (3.23)$$

其中 $\beta \in (0, 1)$ 为动量系数（通常取 0.9）。 $\mathbf{g}_t$ 为反向传播时计算的梯度。动量法的效果如同一个滚下山坡的球体：在平坦方向上持续加速，在陡峭方向上累积的震荡因方向交替而相互抵消，从而加速收敛。

命题 3.14 (NAG (Nesterov Accelerated Gradient))

NAG 是对动量法的改进，其核心思想是“超前看”——先按照当前动量方向走一步，再计算该“未来位置”的梯度：

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla \mathcal{L}(\boldsymbol{\theta}_{t-1} - \eta \beta \mathbf{v}_{t-1}), \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \mathbf{v}_t \quad (3.24)$$

NAG 具有“矫正”作用：当动量方向即将冲过头时，超前梯度能提前修正方向，因此在凸优化中具有更快的收敛保证。实践中，NAG 通常比标准动量法表现更优。

命题 3.15 (自适应学习率方法)

SGD 及其动量变体对所有参数使用统一的学习率，这在处理稀疏特征或不同尺度的参数时效率低下。自适应方法为每个参数单独维护学习率。

AdaGrad：对频繁出现的特征使用较小的学习率，对稀疏特征使用较大的学习率。维护梯度平方的累加和：

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t^2, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t \quad (3.25)$$

优点：无需手动调节学习率，适合稀疏数据。缺点： $\mathbf{s}_t$ 单调递增，学习率最终趋近于 0，导致训练提前停止。

RMSProp：引入指数加权平均替代累加和，解决 AdaGrad 学习率归零的问题：

$$\mathbf{s}_t = \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t^2, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t \quad (3.26)$$

其中 γ 通常取 0.9。RMSProp 在非凸优化中表现稳健，尤其适合 RNN 等动态网络。

AdaDelta：RMSProp 的延伸，进一步去除了对全局学习率 η 的依赖，使用参数更新量的均方根值进行缩放：

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\sqrt{\Delta \boldsymbol{\theta}_{t-1}^2 + \epsilon}}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t \quad (3.27)$$

其中 $\Delta \boldsymbol{\theta}_{t-1}^2$ 为历史更新量的指数加权平均。

命题 3.16 (Adam (Adaptive Moment Estimation))

Adam 是目前最广泛使用的优化器，它融合了动量法（一阶矩）和 RMSProp（二阶矩）的思想，并加入了偏差校正以应对初始阶段矩估计的偏差。

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad (\text{一阶矩：梯度的均值}) \quad (3.28)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \quad (\text{二阶矩：梯度的未中心化方差}) \quad (3.29)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (\text{偏差校正}) \tag{3.30}$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{3.31}$$

Adam 的超参数默认值为： $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ 。

注[优化器选择指南]

- **SGD + Momentum**: 泛化性能通常最佳，尤其适合计算机视觉任务，但需精细调节学习率。
- **Adam**: 收敛快，超参数鲁棒，适合大多数任务，尤其是自然语言处理和生成模型。
- **AdamW**: Adam 的改进版，将权重衰减与梯度更新解耦，在大规模 Transformer 训练（如 BERT、GPT）中成为标配。

3.1.3 应用于图像分类的卷积神经网络

卷积神经网络自 2012 年以来经历了快速的架构演进，每一次突破都深化了我们对深度、宽度、连接模式与训练动态之间关系的理解。本小节系统梳理六个里程碑式架构的设计哲学与核心贡献。

3.1.3.1 AlexNet ——深度学习的破冰者

AlexNet [krizhevsky2012imagenet] 由 Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 于 2012 年提出，在 ImageNet ILSVRC 挑战赛中以 **15.3%** 的 Top-5 错误率夺冠（第二名 26.2%），被公认为深度学习复兴的起点。

3.1.3.1.1 架构设计 AlexNet 包含 8 层可学习参数：5 层卷积层和 3 层全连接层。其核心设计包括：

- **ReLU 激活函数**: 首次在大规模网络中使用 ReLU（而非 Tanh），训练速度提升约 6 倍。ReLU 的稀疏激活特性也缓解了梯度消失问题。
- **局部响应归一化**: 在相邻通道间进行归一化，抑制过大的激活值，增强泛化能力。
- **重叠池化**: 步长小于池化窗口大小（步长 2，窗口 3×3），增加特征图的冗余性。
- **数据增强**: 随机裁剪、水平翻转和颜色扰动，将 120 万张训练图像扩充至 2048 倍。
- **Dropout**: 在全连接层中使用 Dropout（概率 0.5），有效防止过拟合。
- **GPU 并行**: 将网络分布在两块 GTX 580 GPU 上，采用分组卷积策略，突破当时显存限制。

3.1.3.1.2 历史意义与局限 AlexNet 证明了深度（8 层）和大规模数据（ImageNet）结合的有效性，并确立了 GPU 训练的标准范式。其局限在于：全连接层参数量巨大（约 5800 万），占总参数的 95%，导致过拟合风险高且计算冗余。

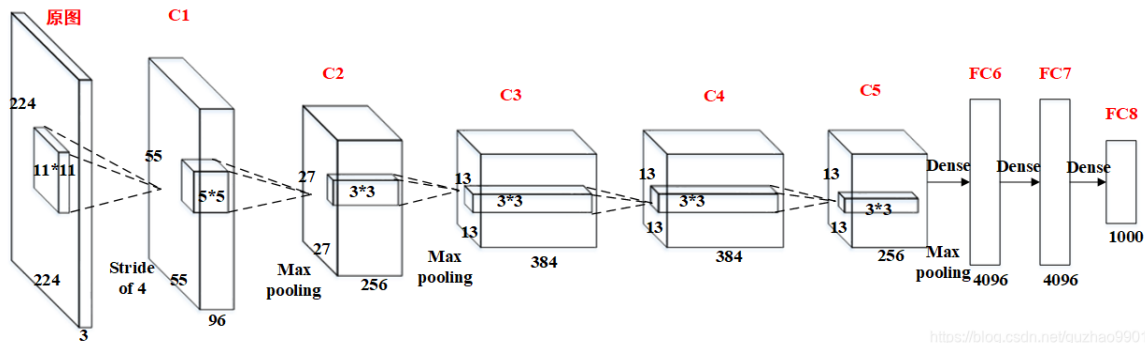


图 3.1: AlexNet 架构示意图（括号内为输出通道数）

3.1.3.2 VGG ——深度与简洁的极致统一

VGGNet [simonyan2014very] 由牛津大学视觉几何组（Visual Geometry Group）于 2014 年提出，在 ILSVRC 2014 中获得定位任务冠军和分类任务亚军（Top-5 错误率 7.3%）。

3.1.3.2.1 核心设计哲学 VGG 的最大贡献在于：用统一的 3×3 卷积核堆叠来替代大卷积核。

- 两层 3×3 卷积堆叠的等效感受野为 5×5 ，参数量为 $2 \times 3^2 = 18$ ，小于单个 5×5 卷积的 $5^2 = 25$ 。
- 三层 3×3 卷积堆叠的等效感受野为 7×7 ，参数量为 $3 \times 9 = 27$ ，远小于单个 7×7 卷积的 49。
- 更深的堆叠引入了更多的非线性变换（每层后接 ReLU），增强了网络的表达能力。

3.1.3.2.2 训练策略与参数初始化 VGG 的深层结构（如 VGG-16/19）在收敛后展现出清晰的层次化特征表达。一个关键的训练技巧是**层次参数继承**：将已收敛模型的前 4 个卷积层和后 3 个全连接层的权重，作为更深神经网络对应层的初始值，而其余卷积层则采用随机初始化。

- **前 4 层卷积**：捕获边缘、纹理等通用底层特征，这些特征在不同视觉任务中具有高度可迁移性，继承其权重可为深层网络提供稳定的梯度流起点。
- **后 3 个全连接层**：承载了高级语义组合的决策映射，若目标任务分类数与源任务一致，保留这部分权重可显著加速收敛并抑制初始损失震荡。
- **中间层随机初始化**：由于深层卷积特征具有任务特异性（如物体部件组合），随机初始化使网络能根据新数据重学从底层到语义的映射，避免源域偏见的干扰。

该策略本质上是**迁移学习中的部分微调**，在 VGG 的原始训练及后续微调任务中被广泛采用，有效缓解了深层网络在数据不足时的过拟合与收敛困难问题。

3.1.3.2.3 局限性与影响

- **参数量巨大**：VGG-16 全连接层参数量高达 1.24 亿（占总数 90%），存储和推理成本极高。
- **训练困难**：19 层网络已经面临梯度消失风险（需配合 He 初始化 [he2015delving] 和精细的学习率调度）。上述层次继承策略一定程度上减轻了极深模型从零训练的难度。
- **架构遗产**：VGG 证明了**网络深度比宽度更重要**，并确立了“小卷积核 + 大深度”的设计范式，影响深远。

3.1.3.3 Inception-v1 (GoogLeNet) ——多尺度卷积的并行探索

Inception-v1 [szegedy2015going] 由 Google 团队于 2014 年提出，获得 ILSVRC 2014 分类任务冠军（Top-5 错误率 6.7%）。其核心创新在于**Inception 模块**——在同一层并行执行不同尺度的卷积。

3.1.3.3.1 Inception 模块的设计动机 传统方法需要人工选择卷积核尺寸（ 3×3 、 5×5 、 7×7 ）。Inception 模块将选择权交给网络：

- 并行执行 1×1 、 3×3 、 5×5 卷积和 3×3 最大池化。
- 将所有输出在通道维度拼接。
- 网络自动学习“在哪个位置使用哪种尺度”最有效。

3.1.3.3.2 1×1 卷积的三大作用 Inception 模块中大量使用 1×1 卷积，其功能远非“逐点卷积”那么简单：

1. **降维**：压缩通道数，大幅降低计算量（通道数从 192 降至 96，减少 50%）。
2. **跨通道信息整合**：在通道维度上做线性组合，增强特征表达。
3. **增加非线性**： 1×1 卷积后接 ReLU，在不改变空间维度的前提下增加网络深度。

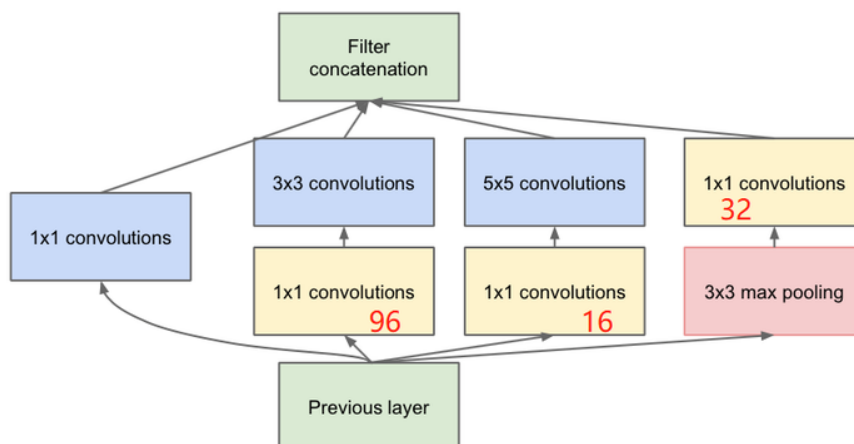


图 3.2: Inception 模块结构示意图 (含 1×1 瓶颈层降维)

3.1.3.3.3 辅助分类器 GoogLeNet 在中间层 (Inception-4a 和 Inception-4d) 添加了两个辅助分类器:

- 将中间层的特征图进行池化、卷积和全连接后输出分类结果。
- 训练时, 辅助损失乘以 0.3 加入总损失。
- 作用: 缓解梯度消失, 提供额外的正则化, 增强低层特征的判别性。

3.1.3.3.4 架构特点总结

- 深度 22 层 (含参数层), 参数量仅约 500 万 (AlexNet 的 1/12)。
- 计算量约 15 亿次浮点运算 (AlexNet 的 1/4)。
- 证明了 更高效的结构设计比单纯堆叠层数更重要。

3.1.3.4 BN-Inception —— 批量归一化带来的训练革命

BN-Inception [ioffe2015batch] 由 Sergey Ioffe 和 Christian Szegedy 于 2015 年提出, 在 Inception-v1 基础上引入 批量归一化 (Batch Normalization), 将 ImageNet 分类准确率提升至 4.8% Top-5 错误率。

3.1.3.4.1 批量归一化的核心思想 批量归一化对每一层的输入进行标准化, 使分布保持稳定:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

其中 μ_B 和 σ_B^2 为 mini-batch 的均值和方差, γ 和 β 为可学习的缩放和偏移参数。

3.1.3.4.2 BN 的五大贡献

1. 解决内部协变量偏移: 每层输入的分布变化得到抑制, 训练更稳定。
2. 允许使用更大的学习率: 梯度被限制在可控范围内, 训练加速。
3. 降低对初始化的敏感性: 即使使用较差的初始化 (如原始 Xavier), 网络仍能收敛。
4. 提供轻微的正则化效果: mini-batch 的随机统计量引入噪声, 类似 Dropout。
5. 允许移除 Dropout: BN-Inception 中部分层移除 Dropout 后, 性能反而提升。

3.1.3.4.3 在 Inception 中的部署

- BN 插入在每个卷积层之后、激活函数之前。
- 使用 BN 后, 学习率从 0.001 提高到 0.01。
- 训练收敛速度提升约 14 倍。
- 模型达到相同精度所需的迭代次数减少 60%。

BN-Inception 标志着 归一化技术成为深度网络训练的标配组件。

3.1.3.5 Inception-v3

3.1.3.6 ResNet